

# **BASELINEPROJECT ENGINEERING DOCUMENT (PED v2.0)**

**SEN381 - MILESTONE 2**

CHRIS DAVID FOURIE – 602826

LISA VERNON – 602006

GROUP E

## Contents

|                                                                                                 |               |
|-------------------------------------------------------------------------------------------------|---------------|
| 1. DOCUMENT CONTROL & REVISION HISTORY .....                                                    | 3             |
| 1.1 Provenance and Research Continuity Across A1, A2, and A3 .....                              | 4             |
| 2. GOVERNING PRINCIPLES & RELATIONSHIP TO MASTER BRIEF .....                                    | 5             |
| 2.1 Compliance with "Apply, Do Not Repeat" (Master Project Brief Section 1).....                | 5             |
| 2.2 The Four Structural Failure Mechanisms of Deferring Decisions (A1 Synthesis) .....          | <b>Error!</b> |
| <b>Bookmark not defined.</b>                                                                    |               |
| 3. PROBLEM ANALYSIS & STAKEHOLDER CONFLICT RESOLUTION (BASELINED M1).....                       | 5             |
| 3.1 Operational Breakdowns in Existing Workflows .....                                          | 5             |
| 3.2 Stakeholder Conflict Resolution Matrix .....                                                | 6             |
| 4. SCOPE BASELINE & BOUNDARY CONTROL.....                                                       | 6             |
| 5. BASELINED REQUIREMENTS & ACCEPTANCE CRITERIA .....                                           | 7             |
| 5.1 Functional Requirements Summary (FR-001 to FR-014) .....                                    | 7             |
| 5.2 Six-State Finite State Machine Lifecycle (DEC-004).....                                     | 7             |
| 6. ARCHITECTURALLY SIGNIFICANT REQUIREMENTS (ASRs) & QUALITY DRIVERS .....                      | 8             |
| 7. MACRO-ARCHITECTURE & COMPONENT DECOMPOSITION .....                                           | 9             |
| 7.1 Clean Layered Modular Monolith Architecture .....                                           | 9             |
| 7.2 Proportional Architecture Defence: Explicit Rejection of Microservices .....                | 10            |
| 8. TECHNOLOGY STACK COMMITMENT (RESOLVING ADR-003 VIA ADR-008) .....                            | 11            |
| 8.1 Research Evolution from Assignment 1 to Milestone 2 .....                                   | 11            |
| 8.2 Empirical Weighted Decision Matrix (ADR-008).....                                           | 11            |
| 9. DATA & PERSISTENCE ARCHITECTURE (STRICT 3NF & CONCURRENCY CONTROL) .....                     | 12            |
| 9.1 Relational Schema & Entity-Relationship Architecture (DOC-ARCH-DATA-001) .....              | 12            |
| 9.2 ACID Boundaries Resolving the Three A2 Failure Modes (ADR-006) .....                        | 12            |
| 9.3 Optimistic Concurrency Control (OCC) vs Pessimistic Locking .....                           | 13            |
| 9.4 Caching Correctness vs Staleness Trade-Off (A2 Synthesis).....                              | 13            |
| 9.5 Schema Evolution Strategy: Expand-Contract Migration Pattern (A3 Synthesis) .....           | 14            |
| 10. RESEARCH-INFORMED INITIAL DESIGN PATTERNS .....                                             | 14            |
| 10.1 Design Problem 1: Decoupled Multi-Channel Notifications (Observer Pattern -- ADR-004)..... | 14            |
| 10.2 Design Problem 2: Polymorphic Request Intake (Factory Method Pattern -- ADR-005) .....     | 15            |
| 11. INFORMATION ARCHITECTURE & ACCESSIBLE UI DESIGN .....                                       | 15            |
| 11.1 Role-Based User Workflows.....                                                             | 15            |
| 11.2 Usability & WCAG 2.1 AA Compliance (NFR-003) .....                                         | 15            |
| 12. API CONTRACTS & TRANSACTIONAL OUTBOX INTEGRATION .....                                      | 16            |

|                                                                        |    |
|------------------------------------------------------------------------|----|
| 12.1 Core RESTful Endpoints (OpenAPI 3.0 Contract) .....               | 16 |
| 12.2 Transactional Outbox Pattern (ADR-007) .....                      | 16 |
| 13. DEPLOYMENT, ENVIRONMENT PARITY & OPERATIONAL READINESS .....       | 17 |
| 13.1 Four-Tier Environment Model (FEC-004) .....                       | 17 |
| 13.2 Synthesis of the Three Industry Fallacies (A3 Synthesis) .....    | 17 |
| 13.3 Zero-Cost Hosting & Resource Limits (NFR-010) .....               | 17 |
| 14. REQUIREMENTS TRACEABILITY EVOLUTION (RTM v2.0 SUMMARY) .....       | 18 |
| 14.1 Deep End-to-End Trace Walkthroughs .....                          | 18 |
| 15. UNCERTAINTY MANAGEMENT & PROJECT RISK REGISTER v2.0 .....          | 19 |
| 16. FORWARD ENGINEERING CONSIDERATIONS REGISTER (M3 PREPARATION) ..... | 22 |
| 17. MASTER ENGINEERING DECISION LOG & ADR SUMMARY .....                | 22 |
| 18. BASELINE SIGN-OFF GATE RECORD (APPENDIX D CONFORMING) .....        | 23 |
| 19. MEANINGFUL DEVELOPMENT & REPOSITORY EVIDENCE (CRITERION F) .....   | 24 |
| 20. ACADEMIC REFERENCES & STANDARDS CITATIONS .....                    | 27 |

# DOCUMENT METADATA & INSTITUTIONAL CONTROL

- **Academic Institution:** Belgium Campus ITiversity
- **Module Code & Name:** SEN381 -- Software Engineering 381 (NQF Level 8)
- **Project Name:** CivicConnect (Community Service Request Management Platform)
- **Document Reference Identifier:** DOC-PED-002 (Controlled Architecture Baseline)
- **Baseline Version:** 2.0 (Supersedes DOC-PED-001 v1.0 Approved on 2026-09-09)
- **Release Date:** 30 September 2026
- **Target Delivery Gate:** Milestone 2 Assessment Gate (Appendix D Conforming)
- **Assessment Weight:** 50 Raw Marks (25 Project Marks) -- 30 Shared Team Marks + 20 Individual Examination Marks
- **Registered Engineering Team (Group E):**
  - **Chris Fourie (Student ID: 602826)** -- Systems Architect, Persistence & Governance Lead (~80% Workload Allocation)
  - **Lisa Verson (Student ID: 602006)** -- Lead Requirements, UI/UX & Design Analyst (~20% Workload Allocation)
  - *(Former Member Note: Pandora Greyling [Student ID: 602369] formally withdrew from campus on 2026-09-29 following 0% contribution in Assignment 3. All responsibilities, risks, and defence topics were formally absorbed and reallocated under ADR-009 and RSK-011).*

## 1. EXECUTIVE SUMMARY & RUBRIC EVALUATION CROSS-REFERENCE

Milestone 2 transitions CivicConnect from the initial requirements baseline established in Milestone 1 into a controlled, defensible, and verified architectural foundation. In strict compliance with the **SEN381 Master Project Brief (Sections 18, 20.2)** and the **Milestone 2 Brief (Sections 4 to 15)**, this document does not present a disconnected "report" or speculative code scaffolding. Instead, it evolves the single Project Engineering Document into **PED v2.0**, combining empirical research evidence from Assignment 1, Assignment 2, and Assignment 3 with practical, test-verified software construction.

## 1. DOCUMENT CONTROL & REVISION HISTORY

| Version | Date       | Primary Authors / Contributors              | Reviewer(s) & Approval Authority | Baseline Status         | Summary of Milestone Scope & Evolution                                               |
|---------|------------|---------------------------------------------|----------------------------------|-------------------------|--------------------------------------------------------------------------------------|
| 1.0     | 2026-09-09 | Chris Fourie, Pandora Greyling, Lisa Verson | Full Team (Two-Reviewer Policy)  | APPROVED<br>M1 BASELINE | Established initial problem framing, 6 operational breakdowns, stakeholder personas, |

|            |            |                                                                             |                                                  |                               |                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------|------------|-----------------------------------------------------------------------------|--------------------------------------------------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            |            |                                                                             |                                                  |                               | scope baseline, 14 FRs, 10 NFRs, 6-state ticket lifecycle FSM, initial RTM v1.0, Risk Register v1.0, and ADR-001 to ADR-003.                                                                                                                                                                                                                                                                                                   |
| <b>1.5</b> | 2026-09-18 | Lisa Verson, Pandora Greyling                                               | Chris Fourie                                     | Architecture Draft            | Integrated empirical research findings from Assignment 2 (Tasks 1 to 5): Observer pattern, Factory Method pattern, relational 3NF draft, and optimistic concurrency comparison.                                                                                                                                                                                                                                                |
| <b>2.0</b> | 2026-09-30 | Chris Fourie, Lisa Verson<br><i>(with early M2 inputs from P. Greyling)</i> | Chris Fourie & Lisa Verson<br>(ADR-009 Ratified) | <b>CONTROLLED M2 BASELINE</b> | Formal M2 Baseline: 7 ASRs, Clean Layered Macro-Architecture, Weighted Tech Stack Matrix (ADR-008), Strict 3NF Relational Model (ADR-006), Design Patterns (ADR-004, ADR-005), Transactional Outbox (ADR-007), OpenAPI 3.0 specs, WCAG 2.1 AA UI specs, RTM v2.0, Risk Register v2.0, 2-Person Emergency Governance Restructuring (ADR-009), Working Codebase (16 passing tests), and Appendix D Sign-Off ( <b>ACCEPTED</b> ). |

## 1.1 Provenance and Research Continuity Across A1, A2, and A3

CivicConnect represents an unbroken chain of evidence-based engineering progression across three formal research deliverables:

- Assignment 1 (Research Foundations):** Group E established the foundational principles of software engineering beyond programming, analyzing the six core engineering concerns (Stakeholder needs, Scope, Schedule, Cost/resources, Quality, Risk) and examining the Boeing 737 MAX MCAS failure (Johnston & Harris, 2019; Leveson, 2020) as an empirical warning against schedule-driven cutting of safety and

documentation. A1 established Boehm's Cost of Change curve (Boehm, 1981) and evaluated initial candidate technology stacks (Stack A: .NET 8 / C# vs Stack B: MERN).

3. **Assignment 2 (Research to Engineering Decisions):** Group E investigated concrete design patterns, persistence correctness, API integration, and collaborative CI controls. A2 evaluated the Observer pattern for status notifications (resolving Design Problem 1), the Factory Method pattern for polymorphic request intake (resolving Design Problem 2), Optimistic Concurrency Control (OCC) for multi-user ticket assignment, and the Transactional Outbox pattern for external citizen messaging.
4. **Assignment 3 (Research for Quality, Security & Production Readiness Decisions):** Group E formulated the formal quality strategy (QA vs QC vs V&V under IEEE SWEBOOK v3.0 and ISO/IEC 29119), threat modeling under STRIDE (Shostack, 2014), defense-in-depth security controls (defeating BOLA/IDOR and SQL injection), production environment parity (12-Factor App Factor X), and the Expand-Contract database migration pattern. A3 also formally registered the non-participation of Pandora Greyling (602369, 0%), establishing the empirical basis for our Milestone 2 emergency governance restructuring under ADR-009.

## 2. GOVERNING PRINCIPLES & RELATIONSHIP TO MASTER BRIEF

### Compliance with "Apply, Do Not Repeat"

In strict accordance with **SEN381 Master Project Brief Section 1**, this document deliberately refrains from reciting textbook definitions of design patterns, normal forms, or architectural styles. Instead, every section applies engineering standards (ISO/IEC/IEEE 29148 for requirements, ISO/IEC 25010 for software quality, IEEE 1016 for design descriptions) directly to CivicConnect's operational constraints, demonstrating *why* decisions were made, *what* trade-offs were accepted, and *how* verified code translates those decisions into practice.

## 3. PROBLEM ANALYSIS & STAKEHOLDER CONFLICT RESOLUTION (BASELINED M1)

### 3.1 Operational Breakdowns in Existing Workflows

CivicConnect replaces fragmented, informal communication channels (WhatsApp groups, paper logs, unshared spreadsheets) that previously resulted in:

1. **Ticket Loss & Duplication:** Identical municipal faults logged repeatedly across separate sheets without deduplication.
2. **Zero Citizen Tracking:** Requesters received no confirmation receipt, tracking identifier, or automated progress alerts.
3. **Queue Ambiguity:** Technicians lacked clear task queues, resulting in dropped tickets or conflicting double-assignments.

4. **Unaccountable Status Changes:** Statuses altered verbally without non-repudiable audit logs of authorization.
5. **Lack of Performance Telemetry:** Supervisors could not track resolution velocity, overdue tickets, or SLA compliance.
6. **POPIA Privacy Violations:** Unencrypted citizen contact details shared across public community chat groups.

### 3.2 Stakeholder Conflict Resolution Matrix

| Stakeholder Persona                          | Core Operational Priority                          | Systemic Conflict Surface                                                                        | Engineering Resolution in Milestone 2                                                                                                                       |
|----------------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Community Requester (Dr. S. Khumalo)</b>  | Sub-2-minute submission; live tracking visibility. | Desires frequent notifications, but manual messaging creates unsustainable staff burden.         | <b>Observer Pattern (ADR-004)</b> and <b>Transactional Outbox (ADR-007)</b> automate SMS/email alerts upon state changes without staff manual intervention. |
| <b>Field Technician (J. Sithole)</b>         | Unambiguous queue; clear location; mobile clarity. | Needs fault data but must not be distracted by citizen chat or exposed to citizen PII liability. | <b>Factory Method Pattern (ADR-005)</b> validates mandatory structural data; DTO masks citizen contact details on technician screens (NFR-005).             |
| <b>Department Supervisor (M. Patel)</b>      | Dynamic triage; fast assignment; SLA enforcement.  | Multiple supervisors assigning tickets concurrently causes race conditions and lost updates.     | <b>Optimistic Concurrency Control (ADR-006)</b> with integer version checking prevents double-assignment collisions and returns HTTP 409 Conflict.          |
| <b>Compliance Officer (R. Van Der Merwe)</b> | Complete auditability; strict POPIA data privacy.  | Full audit logging must not introduce query latency bottlenecks to standard API paths.           | Append-only service_request_audit_logs table committed in atomic ACID transactions; queries execute against non-blocking read replicas.                     |

## 4. SCOPE BASELINE & BOUNDARY CONTROL

- **Committed In-Scope Capabilities (Milestones 1 to 4):**
  - FR-001 to FR-014: Citizen fault submission, 5-category taxonomy, tracking reference generation (REQ-YYYY-NNNN), role-based queues, technician assignment, 6-state FSM state enforcement, resolution recording with

mandatory action notes, executive analytics dashboard, SLA performance tracking, and CSV data export.

- **Deliberately Deferred Capabilities (Justified in ADR-001):**
  - Multi-language UI localization (Deferred to Post-M4).
  - Direct WhatsApp conversational chatbot integration (Deferred to Post-M4 due to third-party API licensing costs violating NFR-010).
  - Live GPS vehicle telematics tracking (Deferred to Post-M4).
- **Explicitly Out-of-Scope Exclusions:**
  - Municipal billing/tariff collection systems, council procurement systems, physical hardware sensors.

## 5. BASELINED REQUIREMENTS & ACCEPTANCE CRITERIA

### 5.1 Functional Requirements Summary (FR-001 to FR-014)

- FR-001 -- Citizen Service Request Submission with dynamic category validation.
- FR-002 -- Multi-Category Intake Classification (FAC\_FAULT, IT\_SUPPORT, SECURITY\_HAZARD, GENERAL\_MAINT, LOST\_PROPERTY).
- FR-003 -- Public Tracking Status Lookup via unique reference number.
- FR-004 -- Authenticated Requester Submission History.
- FR-005 -- Automated Multi-Channel Citizen Status Notifications.
- FR-006 -- Role-Based Operational Work Queues (STAFF, SUPERVISOR, ADMIN).
- FR-007 -- Multi-Criteria Ticket Filtering, Sorting, and Pagination.
- FR-008 -- Request Detail Inspection with POPIA Citizen PII Masking.
- FR-009 -- Technician Ticket Assignment and Ownership Claiming.
- FR-010 -- Server-Side Finite State Machine Status Transition Integrity.
- FR-011 -- Mandatory Resolution Evidence Logging (Action Notes and Actor ID).
- FR-012 -- Executive Service Activity Analytics Dashboard.
- FR-013 -- SLA Overdue Calculation and Priority Performance Metrics.
- FR-014 -- Structured Reporting and Historical Data Export (CSV).

### 5.2 Six-State Finite State Machine Lifecycle (DEC-004)

The platform enforces a deterministic, server-side validated state transition:

- **Guard Rules:**

1. A ticket cannot skip intermediate states (e.g., SUBMITTED directly to RESOLVED is strictly rejected with HTTP 400).
2. Transitioning to RESOLVED or CLOSED enforces mandatory non-empty resolution notes (FR-011).
3. Reversible workflows are supported where justified (e.g., RESOLVED back to IN\_PROGRESS if QA verification fails).

## 6. ARCHITECTURALLY SIGNIFICANT REQUIREMENTS (ASRs) & QUALITY DRIVERS

In Milestone 2, architecture is shaped specifically by the quality attributes and constraints that enforce systemic boundaries:

| ASR Identifier                               | Driving NFR / FR | Quantitative Threshold / Target                                                  | Architectural Mechanism in Milestone 2                                                                                              |
|----------------------------------------------|------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>ASR-001 (Latency &amp; Throughput)</b>    | NFR-001, FR-001  | p95 server response time <= 500ms under 50 concurrent active users.              | Relational B-Tree index optimization; asynchronous non-blocking Node.js event loop; in-memory caching of taxonomy tables.           |
| <b>ASR-002 (Zero-Cost Sustainability)</b>    | NFR-010          | \$0.00/month operational spend; container memory <= 512MB RAM.                   | Selection of lightweight Node.js Alpine runtime (<180MB RAM peak in ADR-008); rejection of standalone JVM message brokers.          |
| <b>ASR-003 (Integrity &amp; Concurrency)</b> | NFR-009, FR-009  | Zero lost updates during concurrent ticket claiming; 100% referential integrity. | Strict 3NF PostgreSQL schema with foreign key constraints; Optimistic Concurrency Control using integer version checking (ADR-006). |
| <b>ASR-004 (Lifecycle Non-Repudiation)</b>   | NFR-006, FR-010  | 100% immutable capture of actor ID, timestamp, old state, and new state.         | Append-only service_request_audit_logs table written inside the same ACID database transaction as the status update.                |
| <b>ASR-005 (POPIA Citizen Privacy)</b>       | NFR-005, FR-008  | Zero unauthorized citizen PII exposure; field-level privacy masking.             | Database is_anonymized_display flag; application service DTO masks citizen contact details on operational technician screens.       |
| <b>ASR-006 (Decoupled)</b>                   | FR-005, NFR-002  | External gateway outages must never fail or roll back                            | In-memory <b>Observer Pattern (ADR-004)</b> combined with asynchronous <b>Transactional Outbox (ADR-007)</b> .                      |

|                                      |                |                                                                              |                                                                                                                                         |
|--------------------------------------|----------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Notification Sinks)                  |                | ticket state updates.                                                        |                                                                                                                                         |
| ASR-007 (Extensible Intake Pipeline) | FR-001, FR-002 | Adding new municipal categories requires zero modification to intake routes. | <b>Factory Method Pattern (ADR-005)</b> isolates category-specific validation into independent creator classes (Open/Closed Principle). |

## 7. MACRO-ARCHITECTURE & COMPONENT DECOMPOSITION

### 7.1 Clean Layered Modular Monolith Architecture

CivicConnect adopts a Clean / Layered Architecture with strict inward dependency inversion (Martin, 2018; Parnas, 1972). To ensure high cohesion and loose coupling across components, the system is decomposed into four discrete architectural layers:

- **1. Presentation Layer (External Interface & Delivery):**
  - **Components:** React 18 web client UI (styled with Tailwind CSS and WCAG 2.1 AA accessible tokens), Express.js REST API controllers (RequestController), route handlers, and centralized HTTP error-handling middleware.
  - **Responsibilities:** Accepts HTTP requests, parses and sanitizes incoming JSON payloads, validates client authentication claims via JWT bearer tokens, delegates execution to Application use cases, and formats outgoing responses adhering to RFC 7807 problem details.
  - **Dependencies:** Depends strictly inward on the Application Services Layer. It has zero direct awareness of database schemas or persistence logic.
- **2. Application Services Layer (Use Cases & Workflow Coordination):**
  - **Components:** Business use cases (CreateServiceRequest, AssignServiceRequest, UpdateServiceRequestStatus, ResolveServiceRequest), DTO mappers (ServiceRequestDTOMapper), and asynchronous event coordinators.
  - **Responsibilities:** Orchestrates domain entities to satisfy business transactions, executes server-side role-based authorization guards, coordinates the in-memory Observer Pattern (DomainEventDispatcher), and manages the Transactional Outbox worker (TransactionalOutboxService).
  - **Dependencies:** Depends strictly on the Domain Core Layer and interface abstractions.
- **3. Domain Core Layer (Enterprise Business Rules & Invariants):**

- **Components:** Core domain entities (ServiceRequest), aggregates, domain event definitions (ServiceRequestStatusChangedEvent), category creation factories (IServiceRequestFactory, CategoryFactories), and repository interface abstractions (IServiceRequestRepository).
  - **Responsibilities:** Encapsulates core municipal business logic, enforces the 6-state ticket lifecycle Finite State Machine (DEC-004), enforces category-specific polymorphic validation rules (Open/Closed Principle), and defines transactional integrity contracts.
  - **Dependencies:** Completely independent and framework-agnostic. The Domain Core contains zero external library dependencies, database drivers, or web framework imports.
- **4. Infrastructure / Data Layer (Persistence & Adapters):**
    - **Components:** PostgreSQL 16 relational database adapters (PostgresServiceRequestRepository, InMemoryServiceRequestRepository), SQL migration scripts, Docker Compose orchestration (DEC-005), and external communication gateway adapters (Email / SMS dispatchers).
    - **Responsibilities:** Implements the persistence interfaces defined by the Domain Core, manages database connection pools, executes atomic transactions under Read Committed isolation, and enforces Optimistic Concurrency Control (OCC) version verifications.
    - **Dependencies:** Points inward to implement Domain repository interfaces via the Dependency Inversion Principle (DIP). Database technologies can be refactored or replaced without modifying core business rules.

## 7.2 Proportional Architecture Defence: Explicit Rejection of Microservices

In accordance with **Milestone 2 Brief Section 5.3** and **Assignment 2 Task 3**, a distributed microservices architecture was evaluated and explicitly rejected based on L. Peter Deutsch's (1994) *Eight Fallacies of Distributed Computing*:

1. **Fallacy 1 (The network is reliable) & Fallacy 2 (Latency is zero):** Splitting CivicConnect into independent microservices (Auth, Intake, Queue, Notification, Analytics) introduces 5ms to 50ms of network latency per hop and packet serialization tax, directly violating our sub-500ms p95 latency budget (ASR-001).
2. **Fallacy 4 (The network is secure):** Distributing services forces mutual TLS authentication, complex API gateway routing, and distributed secret management across multiple process boundaries.
3. **The Distributed Transaction Tax:** Microservices require complex distributed 2PC or Saga orchestrators to maintain consistency across requests, audit ledgers, and notification queues, introducing partial failure modes.
4. **Severe Free-Tier Resource Breach (ASR-002):** Running 5 distinct microservice containers requires over 1.5GB of RAM, immediately breaching the free-tier 512MB RAM cap on cloud PaaS platforms and triggering Out-Of-Memory (OOM) termination.

5. **Team Capacity:** Building, testing, and operating a distributed service mesh in a 2-person student team within 7 weeks represents unwarranted over-engineering.
- **Conclusion: A Clean Layered Modular Monolith** achieves identical logical decoupling, strict information hiding (Parnas, 1972), and isolated unit testability while operating at <180MB RAM with zero distributed latency overhead.

## 8. TECHNOLOGY STACK COMMITMENT (RESOLVING ADR-003 VIA ADR-008)

### 8.1 Research Evolution from Assignment 1 to Milestone 2

In Assignment 1 (Section 3), Group E evaluated candidate technology stacks, initially scoring Stack A (.NET 8 / C# / Microsoft SQL Server on Azure) at 8.48/10 and Stack B (MERN / MongoDB on AWS) at 7.15/10. Stack A was favored in A1 due to C#'s enterprise static typing, built-in security middleware, and long-term Microsoft support.

However, when transitioning from conceptual research to concrete engineering in Milestone 2, the team evaluated candidate stacks against the hard operational constraint of **NFR-010 (R0.00/month cloud hosting budget and the strict 512MB RAM container cap on free-tier cloud PaaS platforms)**.

- Under these binding constraints, ASP.NET Core 8 + SQL Server was disqualified because its base runtime idles at 250MB to 350MB of RAM, leaving negligible memory headroom under the 512MB container limit, and Microsoft SQL Server cannot run inside free-tier cloud containers.
- MongoDB was disqualified because document stores provide BASE eventual consistency rather than the strict ACID guarantees mandated by ASR-003 and ASR-004.
- To capture the compile-time static type safety and OOP modularity of Stack A while adhering to the free-tier memory envelope, the team synthesized **TypeScript 5.3 + Node.js 20 LTS + PostgreSQL 16**. This achieved the static type guarantees of .NET, enforced strict 3NF relational data integrity, and operated inside an ultra-lightweight 45MB-180MB RAM footprint, scoring **9.05 / 10** in ADR-008.

### 8.2 Empirical Weighted Decision Matrix (ADR-008)

| Evaluation Criterion                               | Weight | Candidate A: TypeScript / Node.js | Candidate B: C# / ASP.NET Core 8 | Candidate C: Python / FastAPI |
|----------------------------------------------------|--------|-----------------------------------|----------------------------------|-------------------------------|
| Free-Tier Quota & Memory Footprint (<= 512MB RAM)  | 20%    | 9.0 (1.80)                        | 6.0 (1.20)                       | 7.5 (1.50)                    |
| Architecture & NFR Fit (Typing, Modularity, DTOs)  | 25%    | 9.0 (2.25)                        | 9.5 (2.38)                       | 8.0 (2.00)                    |
| Team Velocity & Learning Curve (2-Person Delivery) | 20%    | 9.5 (1.90)                        | 7.0 (1.40)                       | 7.5 (1.50)                    |

|                                                          |             |                                       |                  |                  |
|----------------------------------------------------------|-------------|---------------------------------------|------------------|------------------|
| <b>Automated Testing &amp; Mocking Tooling Maturity</b>  | 15%         | <b>9.0</b> (1.35)                     | 9.0 (1.35)       | 8.5 (1.28)       |
| <b>Docker Parity &amp; Build Efficiency (Image Size)</b> | 10%         | <b>9.0</b> (0.90)                     | 7.5 (0.75)       | 8.0 (0.80)       |
| <b>Ecosystem Stability &amp; Security Maintenance</b>    | 10%         | <b>8.5</b> (0.85)                     | 9.0 (0.90)       | 8.5 (0.85)       |
| <b>TOTAL WEIGHTED SCORE</b>                              | <b>100%</b> | <b>9.05 / 10</b><br><b>(SELECTED)</b> | <b>7.98 / 10</b> | <b>7.93 / 10</b> |

## 9. DATA & PERSISTENCE ARCHITECTURE (STRICT 3NF & CONCURRENCY CONTROL)

### 9.1 Relational Schema & Entity-Relationship Architecture (DOC-ARCH-DATA-001)

The persistence model enforces strict Third Normal Form (3NF) across 8 normalized tables in PostgreSQL 16:

- roles (role\_id, role\_name, description)
- departments (department\_id, code, name, sla\_escalation\_email)
- users (user\_id, email, password\_hash, role\_id, full\_name, phone\_number)
- staff\_profiles (staff\_id, user\_id, department\_id, is\_on\_duty)
- priorities (priority\_id, code, name, sla\_target\_hours, weight)
- request\_categories (category\_id, code, name, department\_id, default\_priority\_id)
- service\_requests (request\_id, tracking\_reference, title, description, category\_id, department\_id, requester\_id, assigned\_staff\_id, priority\_id, status\_id, is\_anonymized\_display, version, submitted\_at, updated\_at)
- service\_request\_audit\_logs (audit\_id, request\_id, actor\_id, previous\_status\_id, new\_status\_id, action\_notes, logged\_at)
- outbox\_messages (message\_id, event\_type, aggregate\_id, payload, status, retry\_count, created\_at, dispatched\_at)
- status\_transition\_rules (rule\_id, from\_status\_id, to\_status\_id, allowed\_role\_id)

### 9.2 ACID Boundaries Resolving the Three A2 Failure Modes (ADR-006)

In Assignment 2 (Task 2), Group E analyzed three concrete persistence failure modes that threaten municipal service operations:

1. **Partial Write Failure (The Broken Ledger Anomaly):** The ticket update succeeds, but audit log insertion crashes midway, leaving an untraced assignment in violation of POPIA Act 4 of 2013 non-repudiation mandates.
2. **Technician Workload Deadlock (Ghost Assignments):** Workload counters increment, but ticket saving aborts, stranding the technician at false maximum capacity.
3. **State Machine Bypass and Illegal State Mutation:** Malformed API requests force tickets directly from SUBMITTED to RESOLVED, bypassing triage.

CivicConnect eliminates these risks by encapsulating the entire mutation within an explicit **ACID Unit-of-Work transaction boundary**:

- **Atomicity & Consistency:** The update to service\_requests, insertion into service\_request\_audit\_logs, and insertion into outbox\_messages are bound into an indivisible database transaction (BEGIN ... COMMIT). If any operation fails, the database automatically triggers ROLLBACK.
- **Isolation Level:** Configured to **Read Committed**, which prevents dirty reads while avoiding the heavy locking overhead and query serialisation latency of Serializable isolation.
- **Durability:** Committed transactions are flushed to non-volatile storage via PostgreSQL's Write-Ahead Log (WAL), surviving power losses and container restarts.

### 9.3 Optimistic Concurrency Control (OCC) vs Pessimistic Locking

- **The Lost Update Risk (RSK-002):** Multiple supervisors triaging open tickets concurrently or two technicians claiming the same ticket create race conditions.
- **Rejection of Pessimistic Locking (SELECT FOR UPDATE):** Holding row locks holds open database connection pool slots during client think-time, rapidly exhausting the 20-connection cap of cloud free-tier databases and triggering cascading deadlocks.
- **Optimistic Locking Implementation:** service\_requests includes an integer version column. Updates execute conditionally:

sql

```
UPDATE service_requests
```

```
SET assigned_staff_id = $1, status_id = $2, version = version + 1, updated_at = NOW()
```

```
WHERE request_id = $3 AND version = $expectedVersion;
```

If zero rows match, another transaction modified the ticket. The transaction aborts and emits **HTTP 409 Conflict**, prompting the client UI to refresh.

### 9.4 Caching Correctness vs Staleness Trade-Off (A2 Synthesis)

- **Active Ticket Operational Records (STRICTLY NEVER CACHED):** Ticket status, technician assignment, and resolution fields are highly volatile. Caching them in Redis or client memory creates a dangerous stale-read window where supervisors act on outdated data, causing duplicate dispatches.

- **Static Reference Data (JUSTIFIED CACHING):** Service categories, department lookups, priority levels, and SLA targets change infrequently. Caching these tables in application memory with a Time-To-Live (TTL) of 60 minutes reduces database query load without compromising operational correctness.

## 9.5 Schema Evolution Strategy: Expand-Contract Migration Pattern (A3 Synthesis)

In accordance with Assignment 3 (Section 3.5.2), CivicConnect avoids destructive in-place schema migrations that cause downtime or block rollbacks. Future schema modifications follow the **Expand-Contract (Parallel Run) Pattern**:

1. **Expand Phase:** Add new columns as nullable or with defaults (e.g., adding `requester_uuid` alongside `requester_id`) without modifying existing columns.
2. **Parallel Run Phase:** Application v2.0 dual-writes to both columns, reading from the old column. Rolling back to v1.0 remains 100% safe.
3. **Backfill & Swap:** Asynchronous background tasks copy legacy data to the new structure; application v2.1 reads exclusively from the new column.
4. **Contract Phase:** Final migration drops obsolete legacy columns once v2.1 is fully stable.

# 10. RESEARCH-INFORMED INITIAL DESIGN PATTERNS

## 10.1 Design Problem 1: Decoupled Multi-Channel Notifications (Observer Pattern -- ADR-004)

- **Problem & Context:** Modifying a ticket status must notify the citizen (FR-005), alert technicians (FR-006), log audit records (NFR-006), and calculate SLA timers (FR-013). Directly invoking email or SMS services inside the core `ServiceRequest` entity tightly couples domain logic to volatile third-party networks, causing tickets to fail if an external email gateway times out (Saxena and Kumar, 2012; Sarcar, 2021).
- **Research Evidence:** Assignment 2 Task 1 evaluated procedural calls vs in-memory Observer vs distributed message brokers (Kafka/RabbitMQ). Kafka was rejected due to 1GB+ RAM footprints violating NFR-010.
- **Milestone 2 Decision:** Implemented the in-memory **Observer Pattern with Domain Event Dispatcher**. When a transition occurs, `ServiceRequest` publishes a `ServiceRequestStatusChangedEvent`. Registered observers (`NotificationDispatchObserver`, `AuditLoggingObserver`) process events independently.
- **Trade-Off & Introduced Complexity:** Introduces indirect control flow. Mitigated by wrapping observer executions in isolated try/catch error containment blocks so that a failed email notification cannot corrupt the database transaction.

## 10.2 Design Problem 2: Polymorphic Request Intake (Factory Method Pattern -- ADR-005)

- **Problem & Context:** CivicConnect processes heterogeneous municipal categories with conflicting validation rules: Facilities faults require building/room numbers; IT support tickets require hardware asset tags; Security hazards require mandatory emergency priority escalation (FR-001, FR-002). Handling this via monolithic switch/case statements violates the Open/Closed Principle (OCP).
- **Research Evidence:** Assignment 2 Task 1 evaluated Factory Method vs Abstract Factory vs runtime reflection.
- **Milestone 2 Decision:** Implemented the GoF **Factory Method Pattern**. An abstract interface `IServiceRequestFactory` is implemented by 5 concrete creator classes (`FacilitiesRequestFactory`, `ITSupportRequestFactory`, `SecurityHazardRequestFactory`, `GeneralMaintenanceRequestFactory`, `LostPropertyRequestFactory`) managed by a singleton `ServiceRequestFactoryRegistry`.
- **Trade-Off & Introduced Complexity:** Class proliferation (an interface, a registry, and 5 creators instead of a 20-line switch block). Accepted because each category is completely isolated and independently testable. Adding a new category requires zero changes to existing intake handlers.

# 11. INFORMATION ARCHITECTURE & ACCESSIBLE UI DESIGN

## 11.1 Role-Based User Workflows

1. **Community Requester Journey:** 3-step submission modal (Category Selection -> Fault Description/Photo Upload -> Instant Reference Generation REQ-YYYY-NNNN) followed by a live tracking lookup portal.
2. **Field Technician Journey:** Filtered department queue prioritized by SLA deadline, featuring quick-action drawers to claim tickets, add progress notes, and submit resolution evidence.
3. **Department Supervisor Journey:** Comprehensive operational dashboard with live status distributions, overdue alert flags, and technician assignment controls.

## 11.2 Usability & WCAG 2.1 AA Compliance (NFR-003)

- **Contrast Ratios:** All UI text maintains a minimum contrast ratio of 4.5:1 against backgrounds (Tailwind Slate-900 #0F172A on pure White #FFFFFF; status badges pair high-contrast colors with clear text labels).
- **Keyboard Navigation:** All modal windows trap focus for Tab navigation; form fields incorporate visible focus rings (focus:ring-2 focus:ring-blue-600); all interactive elements are reachable without a mouse.
- **Screen Reader Accessibility:** All input fields feature explicit `<label for="...">` markup; dynamic status updates utilize `aria-live="polite"` regions.

- **POPIA Contact Protection:** Citizen phone numbers and emails are automatically masked ("[POPIA PROTECTED]") on operational technician screens, while visible to compliance administrators (NFR-005).

## 12. API CONTRACTS & TRANSACTIONAL OUTBOX INTEGRATION

### 12.1 Core RESTful Endpoints (OpenAPI 3.0 Contract)

All API endpoints communicate via JSON over HTTPS, using standard HTTP status codes (Fielding, 2000; Nottingham and Wilde, 2016):

- GET /health/live -- Returns HTTP 200 { status: "UP", memoryUsageMB: 48, uptimeSeconds } (NFR-002).
- GET /health/ready -- Returns HTTP 200 { status: "READY", database: "CONNECTED" }.
- POST /api/v1/requests -- Citizen creates a service request; validates via Factory Method; returns HTTP 201 Created with tracking reference.
- GET /api/v1/requests -- Returns paginated, filtered ticket collection (status, departmentId, page, limit).
- GET /api/v1/requests/:id -- Retrieves full ticket details with POPIA masking applied for technicians (FR-008).
- PATCH /api/v1/requests/:id/assign -- Assigns technician; validates integer version; returns **HTTP 409 Conflict** on version mismatch (ADR-006).
- PATCH /api/v1/requests/:id/status -- Executes state transition; returns **HTTP 409 Conflict** on stale version, **HTTP 400 Bad Request** on invalid FSM jump or missing FR-011 resolution notes.

### 12.2 Transactional Outbox Pattern (ADR-007)

- **The Dual-Write Hazard:** If a backend route saves a ticket and then synchronously calls SendGrid/Twilio, an external network timeout blocks the user's thread (violating the 500ms latency target), and external gateway crashes leave the database in an inconsistent state.
- **Outbox Implementation:** State changes write both the ticket update and an outbox record into outbox\_messages within the *same atomic database transaction*. An asynchronous background polling worker reads pending outbox messages, transmits them to external providers, and applies exponential backoff retries upon failure.

## 13. DEPLOYMENT, ENVIRONMENT PARITY & OPERATIONAL READINESS

### 13.1 Four-Tier Environment Model (FEC-004)

1. **Local Development (Workstation):** Docker Compose orchestrating PostgreSQL 16 Alpine container and hot-reloading Node.js server.
2. **Automated CI (GitHub Actions):** Operationalized `.github/workflows/pr-governance-check.yml` executing PR validation, traceability checks, and secret scans.
3. **Staging / Demonstration (Cloud PaaS):** Deployed to Render / Vercel connected to a Neon PostgreSQL 16 free-tier database.
4. **Production / Release Evaluation:** Controlled staging environment utilized for formal classroom academic defence.

### 13.2 Synthesis of the Three Industry Fallacies (A3 Synthesis)

In Assignment 3, Group E analyzed three pervasive software engineering fallacies that CivicConnect proactively overcomes:

1. **The Green Pipeline Fallacy (A3 Section 1.5):** A passing automated test suite is not definitive proof of quality; it proves only that the specific inputs and assertions anticipated by the engineer behaved as expected. Passing tests cannot expose semantic gaps, unmodeled business edge cases, or non-functional latency degradation. CivicConnect addresses this through strict human PR peer reviews, negative boundary testing, and realistic integration assertions.
2. **The Clean Scanner Fallacy (A3 Section 2.6):** A security scanner reporting zero high-severity findings proves only that the code lacks known syntactic signatures. Scanners are completely blind to Broken Object-Level Authorization (BOLA / IDOR), session invalidation flaws, or POPIA data leakage. CivicConnect addresses this through tenant-enforced SQL predicates (`WHERE requester_id = :auth_user_id`) and negative authorization integration tests.
3. **The Developer Machine Fallacy (A3 Section 3.8):** A system running cleanly on a local developer laptop provides zero empirical proof of production readiness. Developer machines mask concurrency race conditions, zero-latency loopback illusions, elevated local admin privileges, and environmental configuration drift. CivicConnect eliminates these blind spots using containerized Docker Compose parity (`docker-compose.yml`) and multi-threaded OCC concurrency test suites.

### 13.3 Zero-Cost Hosting & Resource Limits (NFR-010)

- Free-tier cloud PaaS platforms enforce strict container memory limits (512MB RAM cap).
- Our TypeScript/Node.js runtime idles at ~45MB RAM and peaks at <180MB RAM under full test execution, guaranteeing zero Out-Of-Memory (OOM) container crashes without requiring paid cloud infrastructure.

## 14. REQUIREMENTS TRACEABILITY EVOLUTION (RTM v2.0 SUMMARY)

The Requirements Traceability Matrix (DOC-REQ-002) has evolved from an M1 requirements table into a living engineering instrument with all 12 mandatory columns populated across all 24 requirements.

### 14.1 Deep End-to-End Trace Walkthroughs

#### Deep Trace 1: FR-001 (Citizen Service Request Submission)

1. **Requirement ID & Wording:** FR-001 -- Citizen submits a service request with category, description, and location.
2. **Stakeholder & Priority:** Community Requester (Dr. S. Khumalo) -- Priority: Must Have (MoSCoW).
3. **Acceptance Criteria:** AC-001.1 (Valid payload returns HTTP 201 and tracking code); AC-001.2 (Missing location returns HTTP 400).
4. **ASR / Quality Link:** ASR-001 (Latency <= 500ms), ASR-002 (Zero-cost hosting), ASR-007 (Polymorphic intake).
5. **Architecture Layer:** Presentation Layer (RequestController.ts) -> Application Layer (CreateServiceRequest.ts) -> Domain Core (ServiceRequest.ts).
6. **Data / Persistence Impact:** Insert into service\_requests table in V1\_\_initial\_schema.sql with foreign keys to users, departments, priorities.
7. **Design Decision:** Factory Method Pattern (ADR-005) delegating validation to specialized category creators (FacilitiesRequestFactory, etc.).
8. **Technology Decision:** Node.js 20 LTS, TypeScript 5.3, Express, PostgreSQL 16 (ADR-008).
9. **Implementation Evidence:** Source file code/src/application/use-cases/CreateServiceRequest.ts.
10. **Verification Evidence:** Automated unit tests in ServiceRequestFactory.test.ts and integration test in api.test.ts (POST /api/v1/requests returns 201).
11. **Lifecycle Status:** Implemented.
12. **ADR / Change Reference:** ADR-005 (Factory Method), ADR-008 (Tech Stack).

#### Deep Trace 2: FR-010 (Finite State Machine Status Transition Integrity)

1. **Requirement ID & Wording:** FR-010 -- Platform enforces deterministic lifecycle state transitions and prevents illegal status jumps.
2. **Stakeholder & Priority:** Department Supervisor (M. Patel) / Compliance Officer -- Priority: Must Have.
3. **Acceptance Criteria:** AC-010.1 (Valid transition updates status and writes audit log); AC-010.2 (Illegal jump throws error and aborts).

4. **ASR / Quality Link:** ASR-003 (Data Integrity), ASR-004 (Auditability), ASR-006 (Decoupled Notifications).
5. **Architecture Layer:** Presentation (RequestController.ts) -> Application (UpdateServiceRequestStatus.ts) -> Domain (ServiceRequest.ts).
6. **Data / Persistence Impact:** Atomic update to service\_requests.status\_id, insert to service\_request\_audit\_logs, insert to outbox\_messages.
7. **Design Decision:** In-memory Observer Pattern (ADR-004) publishing ServiceRequestStatusChangedEvent; FSM transition matrix (DEC-004).
8. **Technology Decision:** TypeScript custom FSM invariants and PostgreSQL 16 transaction boundaries (ADR-006).
9. **Implementation Evidence:** Method ServiceRequest.transitionToStatus(...) in code/src/domain/entities/ServiceRequest.ts.
10. **Verification Evidence:** Automated unit tests in ServiceRequestFSM.test.ts (4 passed) asserting invalid jumps throw InvalidStateTransitionError.
11. **Lifecycle Status:** Implemented.
12. **ADR / Change Reference:** DEC-004 (FSM Model), ADR-004 (Observer), ADR-006 (Relational OCC).

## 15. UNCERTAINTY MANAGEMENT & PROJECT RISK REGISTER v2.0

Risk Register v2.0 (DOC-RSK-002) tracks 11 project-specific risks evaluated using the quantitative formula Risk Exposure = Likelihood x Impact on a 1-to-5 scale:

| Risk ID | Risk Description & Cause                                                                                         | Pre-RE     | Proactive Architectural Mitigation                                                       | Reactive Contingency Plan                                   | Post-RE   | Status & Owner            |
|---------|------------------------------------------------------------------------------------------------------------------|------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------|---------------------------|
| RSK-001 | <b>Container Out-Of-Memory Crashes:</b><br>Heavy runtimes or message brokers breach the free-tier 512MB RAM cap. | 4 x 5 = 20 | Committed to lightweight Node.js Alpine runtime (<180MB RAM) in ADR-008; rejected Kafka. | Configure Node.js --max-old-space-size=256 memory throttle. | 1 x 3 = 3 | Controlled (Chris Fourie) |
| RSK-002 | <b>Concurrent Assignment Lost Updates:</b>                                                                       | 4 x 4 = 16 | Implemented Optimistic Concurrency Control with                                          | Application catches collision and                           | 1 x 3 = 3 | Controlled (Chris Fourie) |

|                |                                                                                                              |                  |                                                                                                 |                                                                                  |              |                          |
|----------------|--------------------------------------------------------------------------------------------------------------|------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|--------------|--------------------------|
|                | Multiple supervisors assigning the same ticket overwrite changes.                                            |                  | integer version checking in ADR-006.                                                            | returns HTTP 409 Conflict, prompting UI refresh.                                 |              |                          |
| <b>RSK-003</b> | <b>Cascading Gateway Outages:</b> External email/SMS timeouts cause ticket persistence transactions to fail. | 4 x<br>4 =<br>16 | Adopted Observer Pattern (ADR-004) and Transactional Outbox Pattern (ADR-007).                  | Outbox background worker retries asynchronously with exponential backoff.        | 1 x 3<br>= 3 | Controlled (Lisa Verson) |
| <b>RSK-004</b> | <b>Dual-Write Data Inconsistency:</b> Database commits ticket but external notification fails midway.        | 3 x<br>4 =<br>12 | Transactional Outbox commits messages in the same ACID database transaction (ADR-007).          | Poison-message dead letter queue for messages failing after 5 retries.           | 1 x 2<br>= 2 | Controlled (Lisa Verson) |
| <b>RSK-005</b> | <b>Class Proliferation / Indirection:</b> Design patterns introduce excessive classes and complex debugging. | 4 x<br>3 =<br>12 | Consolidated factory classes into a single cohesive domain module; registered in singleton map. | Comprehensive unit tests for factory registry and event dispatcher.              | 2 x 2<br>= 4 | Controlled (Lisa Verson) |
| <b>RSK-006</b> | <b>Unauthorized Citizen PII Leakage:</b> Operational staff view citizen phone numbers, violating POPIA.      | 4 x<br>4 =<br>16 | Automated DTO masking in ServiceRequestDTOMapper intercepts technician views (NFR-005).         | Field-level database encryption and immediate role revocation upon audit breach. | 1 x 3<br>= 3 | Controlled (Lisa Verson) |

|                |                                                                                                                |                  |                                                                                                              |                                                                                                         |              |                              |
|----------------|----------------------------------------------------------------------------------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|--------------|------------------------------|
| <b>RSK-007</b> | <b>Premature Microservices Complexity:</b> Splitting into microservices causes network tax and schedule delay. | 4 x<br>4 =<br>16 | Explicitly rejected microservices in PED Section 7; built Clean Layered Modular Monolith.                    | Modular directory boundaries allow splitting into separate services post-M4 if load warrants.           | 1 x 3<br>= 3 | Controlled<br>(Chris Fourie) |
| <b>RSK-008</b> | <b>Database Single Point of Failure:</b> Free-tier cloud database instance experiences downtime.               | 3 x<br>4 =<br>12 | Fully reproducible containerized PostgreSQL 16 schema in docker-compose.yml (DEC-005).                       | Daily automated SQL logical backup dump to cloud object storage.                                        | 1 x 3<br>= 3 | Controlled<br>(Chris Fourie) |
| <b>RSK-009</b> | <b>Non-Conforming AI Scaffolding:</b> Generative AI produces boilerplate violating project standards.          | 4 x<br>3 =<br>12 | Enforced 5-stage human verification in AI_Usage_Register_v2.0.md; rejected AI suggestions.                   | Reject and rewrite unverified AI code during mandatory PR peer reviews.                                 | 1 x 2<br>= 2 | Controlled<br>(Chris Fourie) |
| <b>RSK-010</b> | <b>Schedule Compression for M2:</b> Delaying decisions causes unverified development before deadline.          | 4 x<br>4 =<br>16 | Resolved ADR-003 early; executed concurrent documentation and construction work streams.                     | Scope triage reducing non-essential UI views while maintaining core domain logic.                       | 1 x 3<br>= 3 | Controlled<br>(Chris Fourie) |
| <b>RSK-011</b> | <b>Team Member Attrition (Capacity Loss):</b> Team member Pandora Greyling (602369)                            | 5 x<br>5 =<br>25 | Enacted Emergency Governance Realignment (ADR-009); Chris Fourie absorbed ~80% workload; Lisa absorbed ~20%. | Adapted review threshold to Single Mandatory Independent Peer Review (100% partner sign-off) + CI Gate. | 1 x 3<br>= 3 | Controlled<br>(Chris Fourie) |

|  |                            |  |  |  |  |  |
|--|----------------------------|--|--|--|--|--|
|  | departed on<br>2026-09-29. |  |  |  |  |  |
|--|----------------------------|--|--|--|--|--|

## 16. FORWARD ENGINEERING CONSIDERATIONS REGISTER (M3 PREPARATION)

In accordance with Assignment 3 (Section 4 Research-to-M3 Engineering Map), the forward engineering considerations directly bridge Milestone 2 architectural baselines into Milestone 3 quality, security, and production readiness activities:

- **FEC-001 (Hardened Authentication & Session Lifecycle):** Transitioning from basic JWT claims to secure refresh token rotation stored in HTTP-only, SameSite cookies in Milestone 3, preventing session hijacking.
- **FEC-002 (Multi-Tier Automated Verification & Mutation Testing):** Overcoming the Green Pipeline Fallacy by expanding unit test suites to mutation testing (Stryker) and end-to-end Supertest integration coverage.
- **FEC-003 (STRIDE Threat Realization & Negative BOLA Integration Tests):** Overcoming the Clean Scanner Fallacy by implementing explicit negative automated integration tests asserting HTTP 403 Forbidden across multi-tenant boundaries (WHERE requester\_id = :auth\_user\_id).
- **FEC-004 (Pre-Commit Gitleaks Scanning & Dynamic Secret Injection):** Operationalizing pre-commit secret detection hooks to prevent accidental credential commits into Git repository history.
- **FEC-005 (Expand-Contract Schema Evolution):** Enforcing the 4-phase Expand-Contract pattern for all relational database schema modifications in Milestone 3.
- **FEC-006 (Three Pillars of Observability & Health Probes):** Integrating structured Winston JSON logging with unique correlation IDs (X-Correlation-ID) across asynchronous outbox lifecycles, and monitoring /health/live and /health/ready endpoints.

## 17. MASTER ENGINEERING DECISION LOG & ADR SUMMARY

- ADR-001 -- Scope Baseline and Change Control Policy (**Accepted**, Milestone 1).
- ADR-002 -- GitHub Governance and Two-Reviewer Approval Policy (**Accepted**, Milestone 1).
- ADR-003 -- Justified Deferment of Technology Stack Selection (**Superseded by ADR-008**, Milestone 2).
- DEC-004 -- Server-Side Finite State Machine Integrity Model (**Accepted**, Milestone 1).

- DEC-005 -- Cloud Free-Tier Deployment with Docker Compose Parity (**Accepted**, Milestone 1).
- ADR-004 -- In-Memory Observer Pattern for Lifecycle Event Notifications (**Accepted**, Milestone 2).
- ADR-005 -- Factory Method Pattern for Polymorphic Request Intake & Validation (**Accepted**, Milestone 2).
- ADR-006 -- Relational Persistence (Strict 3NF) with Optimistic Concurrency Control (**Accepted**, Milestone 2).
- ADR-007 -- Transactional Outbox Pattern for External Gateway Integration (**Accepted**, Milestone 2).
- ADR-008 -- Technology Stack Commitment via Weighted Decision Matrix (**Accepted**, Milestone 2).
- ADR-009 -- Emergency Governance Adjustment & Workload Reallocation for Two-Person Operation (**Accepted**, Milestone 2).

## 18. BASELINE SIGN-OFF GATE RECORD (APPENDIX D CONFORMING)

| Assessment Attribute         | Formal Evaluation Record                                                                                                                                     |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project Name                 | CivicConnect: Community Service Request Management Platform                                                                                                  |
| Baseline Type                | Milestone 2 -- Architecture, Technology & Initial Design Baseline                                                                                            |
| Document Version             | v2.0 (Controlled Architecture Baseline)                                                                                                                      |
| Evaluation Date              | 30 September 2026                                                                                                                                            |
| Scope Reviewed               | <b>YES</b> -- Milestone 1 scope confirmed unchanged; all 14 FRs and 10 NFRs validated against architectural allocations.                                     |
| Architecture & ASRs Checked  | <b>YES</b> -- Proportional Clean Layered Architecture justified; 7 ASRs defined with quantitative metrics; microservices rejected with evidence.             |
| Data & Persistence Checked   | <b>YES</b> -- Strict 3NF relational schema, ERD, declarative constraints, and Optimistic Concurrency Control (ADR-006) verified in PostgreSQL 16 migrations. |
| Technology Selection Checked | <b>YES</b> -- Formally evaluated and committed via Weighted Decision Matrix (ADR-008), resolving ADR-003 under the zero-cost free-tier limit.                |

## Assessment Attribute    Formal Evaluation Record

|                                          |                                                                                                                                                                 |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Design Decisions Checked</b>          | <b>YES</b> -- Two genuine GoF design patterns committed (Observer ADR-004, Factory Method ADR-005, Outbox ADR-007) informed by Assignment 2 research.           |
| <b>Requirements Traceability Checked</b> | <b>YES</b> -- Living RTM v2.0 fully populated across all 12 mandatory columns with active source code, schema, and verification links.                          |
| <b>Risk Review Completed</b>             | <b>YES</b> -- 11 risks evaluated quantitatively in Risk Register v2.0, including team restructuring risk (RSK-011).                                             |
| <b>Controlled Development Checked</b>    | <b>YES</b> -- Meaningful domain models, category creators, event dispatchers, migrations, Docker parity, and 16 passing automated tests verified in repository. |
| <b>Governance &amp; SCM Checked</b>      | <b>YES</b> -- Restructured to 2-person operation under ADR-009 (Single Mandatory Independent Peer Review + Automated CI Gate) following student exit.           |
| <b>GATE OUTCOME</b>                      | <b>ACCEPTED</b>                                                                                                                                                 |

*Formally Ratified Sign-Off Authority:*

- **Lisa Verson (Lead Requirements & Design Analyst -- 602006):** *Signed -- 2026-09-30*
- **Chris Fourie (Systems Architect, Persistence & Governance Lead -- 602826):** *Signed -- 2026-09-30*  
*(Record Note: Pandora Greyling [602369] formally withdrew on 2026-09-29; baseline sign-off ratified under ADR-009 two-person team governance charter).*

## 19. MEANINGFUL DEVELOPMENT & REPOSITORY EVIDENCE (CRITERION F)

### 19.1 Repository Structure & Clean Architecture Alignment

The practical codebase in code/ strictly reflects Clean Architecture boundaries, organizing source code into isolated, cohesive packages with zero circular dependencies:

- **1. Database Persistence Layer (code/database/):**
  - **Migrations (database/migrations/):** Contains V1\_\_initial\_schema.sql, defining the strict 3NF relational schema across all 8 core tables with declarative primary keys, foreign keys (ON DELETE RESTRICT), status enum constraints, and optimistic locking integer versioning.
  - **Seeds (database/seeds/):** Contains 01\_baseline\_seeds.sql, populating baseline administrative roles, municipal department definitions, SLA priority configurations, and valid FSM status transition rules.
- **2. Domain Core Layer (code/src/domain/):**

- **Entities (domain/entities/):** Contains the primary aggregate `ServiceRequest.ts`, which encapsulates the core business invariants, optimistic locking version management, and the 6-state ticket lifecycle Finite State Machine.
- **Enums (domain/enums/):** Defines strongly typed domain constants including `RequestStatus.ts`, `PriorityLevel.ts`, and municipal role specifications.
- **Events (domain/events/):** Contains `IDomainEvent.ts` and `DomainEventDispatcher.ts`, implementing the in-memory Observer Pattern (ADR-004) to decouple state updates from side-effect execution.
- **Factories (domain/factories/):** Contains `IServiceRequestFactory.ts` and `CategoryFactories.ts`, implementing the GoF Factory Method Pattern (ADR-005) for polymorphic category intake and isolated validation.
- **Repositories (domain/repositories/):** Contains interface abstraction `IServiceRequestRepository.ts`, enforcing the Dependency Inversion Principle so business rules remain decoupled from SQL drivers.
- **3. Application Services Layer (code/src/application/):**
  - **Use Cases (application/use-cases/):** Contains orchestrating business workflows including `CreateServiceRequest.ts`, `AssignServiceRequest.ts`, and `UpdateServiceRequestStatus.ts`.
  - **DTOs & Mapping (application/dtos/):** Contains `ServiceRequestDTO.ts`, implementing automated citizen PII masking ("[POPIA PROTECTED]") on operational screens for field technicians (NFR-005).
  - **Observers (application/observers/):** Contains `NotificationDispatchObserver.ts` and `AuditLoggingObserver.ts`, reacting to published domain events without coupling to the core entity.
- **4. Infrastructure Layer (code/src/infrastructure/):**
  - **Outbox Pattern (infrastructure/outbox/):** Contains `TransactionalOutboxService.ts`, implementing the Transactional Outbox Pattern (ADR-007) to eliminate dual-write risks during external notification delivery.
  - **Repository Implementations (infrastructure/repositories/):** Contains `PostgresServiceRequestRepository.ts` for live PostgreSQL queries and `InMemoryServiceRequestRepository.ts` for millisecond unit testing.
- **5. Presentation Layer (code/src/presentation/):**
  - **Controllers (presentation/controllers/):** Contains `RequestController.ts`, translating HTTP REST requests into application use-case calls and mapping concurrency collisions to HTTP 409 Conflict.
  - **Routes (presentation/routes/):** Contains `requestRoutes.ts` and `healthRoutes.ts` (exposing `/health/live` and `/health/ready` probes per NFR-002).
  - **Bootstrap:** `app.ts` configures Express middleware, CORS, and centralized error handling; `server.ts` handles graceful startup and port binding.

- **6. Automated Verification Suites (code/tests/):**
  - **Unit Tests (tests/unit/):** Isolated test suites verifying FSM lifecycle transitions (ServiceRequestFSM.test.ts), Factory Method creators (ServiceRequestFactory.test.ts), Optimistic Concurrency collisions (OptimisticConcurrency.test.ts), Observer event dispatching (ObserverPattern.test.ts), and POPIA data masking (AnonymizationMask.test.ts).
  - **Integration Tests (tests/integration/):** Supertest API tests in api.test.ts executing end-to-end HTTP probes and ticket lifecycle workflows against running Express instances.
- **7. Environment & Configuration:**
  - docker-compose.yml provides containerized PostgreSQL 16 Alpine parity (DEC-005).
  - package.json and tsconfig.json configure strict TypeScript compilation (tsc) and Vitest execution.
  - .env.example isolates configuration and secrets from source code control (Factor III: Config).

## 19.2 Automated Verification Results

All core architectural mechanisms, domain invariants, and API contracts are continuously validated through an automated verification suite executed using Vitest v1.6.1:

- **1. Test Execution Summary:**
  - **Total Test Suites:** 6 test files passed out of 6 (100% success rate).
  - **Total Executed Tests:** 16 tests passed out of 16 (0 failures, 0 skipped).
  - **Execution Performance:** Total test runtime completed in 1.01 seconds, satisfying fast continuous feedback requirements.
- **2. Granular Test Suite Breakdown:**
  - **Optimistic Concurrency Control (OptimisticConcurrency.test.ts):** 2 passed tests verifying that concurrent mutations increment the integer version token, and asserting that a version mismatch immediately aborts and returns an HTTP 409 Conflict status.
  - **Finite State Machine Lifecycle (ServiceRequestFSM.test.ts):** 4 passed tests validating that service requests follow strict linear transitions (SUBMITTED to TRIAGED to ASSIGNED to IN\_PROGRESS to RESOLVED to CLOSED), and confirming that invalid status jumps throw an InvalidStateTransitionError.
  - **Category Factory Method Validation (ServiceRequestFactory.test.ts):** 4 passed tests confirming that category-specific factory creators independently validate distinct mandatory fields (such as building/room numbers for Facilities

and hardware asset tags for IT Support) in compliance with the Open/Closed Principle.

- **Observer Pattern Event Dispatching (ObserverPattern.test.ts):** 1 passed test confirming that state transitions publish a ServiceRequestStatusChangedEvent to registered in-memory observers without coupling external notification side effects to the core entity.
- **POPIA Data Protection Masking (AnonymizationMask.test.ts):** 2 passed tests asserting that citizen contact numbers and email addresses are automatically masked on technician screens while remaining accessible to compliance officers (NFR-005).
- **Supertest REST API Integration (api.test.ts):** 3 passed tests verifying that /health/live and /health/ready endpoints return HTTP 200 OK, and confirming that POST /api/v1/requests successfully creates tickets and generates tracking codes.
- **3. Compilation & Repository Status:**
  - **TypeScript Compiler (tsc):** Clean compilation across the entire project with 0 errors and 0 warnings under strict type-checking settings.
  - **Version Control Alignment:** All source files, migration scripts, and test suites are tracked and synchronized on branch feat/m2-architecture-and-codebase within the team's remote GitHub repository under the governed peer-review workflow (ADR-009).

## 20. ACADEMIC REFERENCES & STANDARDS CITATIONS

1. ACM / IEEE-CS Joint Task Force on Software Engineering Ethics (2020) *Software Engineering Code of Ethics and Professional Practice*. New York / Piscataway, NJ: ACM & IEEE.
2. Al-Yozbaky, R.S., Taha, D.B. and Qasha, R. (2026) 'A stage-oriented attack taxonomy and threat analysis for CI/CD pipelines', *International Journal of Safety and Security Engineering*, 16(6), pp. 1399-1407.
3. Bass, L., Clements, P. and Kazman, R. (2021) *Software Architecture in Practice*. 4th edn. Boston, MA: Addison-Wesley Professional.
4. Bernstein, P.A., Hadzilacos, V. and Goodman, N. (1987) *Concurrency Control and Recovery in Database Systems*. Boston, MA: Addison-Wesley.
5. Beyer, B., Jones, C., Petoff, J. and Murphy, N.R. (2016) *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media.
6. Boehm, B.W. (1981) *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice-Hall.

7. Boehm, B. and Basili, V.R. (2001) 'Software Defect Reduction Top 10 List', *Computer*, 34(1), pp. 135-137.
8. Bosu, A., Greiler, M. and Bird, C. (2015) 'Characteristics of Useful Code Reviews: An Empirical Study at Microsoft', *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*, pp. 146-156.
9. Date, C.J. (2004) *An Introduction to Database Systems*. 8th edn. Boston, MA: Pearson / Addison-Wesley.
10. Deutsch, L.P. (1994) *The Eight Fallacies of Distributed Computing*. Sun Microsystems Technical Report.
11. Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation. University of California, Irvine.
12. Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley.
13. Fowler, M. (2018) *Refactoring: Improving the Design of Existing Code*. 2nd edn. Boston, MA: Addison-Wesley.
14. Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
15. Humble, J. and Farley, D. (2010) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley.
16. IEEE Computer Society (2014) *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide V3.0)*. Piscataway, NJ: IEEE Computer Society Press.
17. IEEE Standards Association (2012) *IEEE Standard for Configuration Management in Systems and Software Engineering (IEEE Std 828-2012)*. New York: IEEE.
18. ISO/IEC (2023) *ISO/IEC 25010:2023 Systems and software engineering -- Systems and software Quality Requirements and Evaluation (SQuaRE) -- Product quality model*. Geneva: International Organization for Standardization.
19. ISO/IEC/IEEE (2018) *ISO/IEC/IEEE 29148:2018 Systems and software engineering -- Life cycle processes -- Requirements engineering*. Geneva: International Organization for Standardization.
20. ISO/IEC/IEEE (2021) *ISO/IEC/IEEE 29119:2021 Systems and software engineering -- Software testing (Parts 1-4)*. Geneva: International Organization for Standardization.
21. Johnston, P. and Harris, R. (2019) 'The Boeing 737 MAX saga: lessons for software organizations', *Software Quality Professional*, 21(3), pp. 4-12.
22. Kleppmann, M. (2017) *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media.
23. Kung, H.T. and Robinson, J.T. (1981) 'On Optimistic Methods for Concurrency Control', *ACM Transactions on Database Systems (TODS)*, 6(2), pp. 213-226.

24. Leveson, N.G. (2020) 'Are We Learning from Software Failures? The Boeing 737 MAX Crashes', *MIT Aeronautics and Astronautics Technical Report*. Cambridge, MA: MIT.
25. Martin, R.C. (2000) 'Design Principles and Design Patterns', *Object Mentor*, 1(1), pp. 1-34.
26. Martin, R.C. (2018) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA: Prentice Hall.
27. Nelson, N., Brindescu, C., McKee, S., Sarma, A. and Dig, D. (2019) 'The life-cycle of merge conflicts: Processes, barriers, and strategies', *Empirical Software Engineering*, 24(5), pp. 2863-2906.
28. NIST (2022) *Secure Software Development Framework (SSDF) Version 1.1*. NIST Special Publication 800-218. Gaithersburg, MD: U.S. Department of Commerce.
29. Nottingham, M. and Wilde, E. (2016) *RFC 7807: Problem Details for HTTP APIs*. Internet Engineering Task Force (IETF).
30. OWASP Foundation (2021) *OWASP Top 10: 2021 -- The Ten Most Critical Web Application Security Risks*. Available at: <https://owasp.org/Top10/>
31. Parnas, D.L. (1972) 'On the criteria to be used in decomposing systems into modules', *Communications of the ACM*, 15(12), pp. 1053-1058.
32. Republic of South Africa (2013) *Protection of Personal Information Act (Act No. 4 of 2013)*. Government Gazette, 581(37067). Cape Town: Government Printer.
33. Richardson, C. (2018) *Microservices Patterns: With examples in Java*. Shelter Island, NY: Manning Publications.
34. Sarcar, S. (2021) *Design Patterns in C#*. 2nd edn. New York: Apress.
35. Saxena, V. and Kumar, P. (2012) 'Coupling & Cohesion Metrics for Object-Oriented Design', *International Journal of Computer Science*, 9(4), pp. 211-216.
36. Shostack, A. (2014) *Threat Modeling: Designing for Security*. Indianapolis, IN: John Wiley & Sons.
37. Wiggins, A. (2017) *The Twelve-Factor App*. Available at: <https://12factor.net/>